

Elevate Labs — SQL Developer Internship

Task 2: Data Insertion and Handling Nulls

Submitted by: **G. Pavana Deepika** | Key Concepts: DML, Null Handling

Part 1: Theoretical & Conceptual Questions

1. What is the difference between NULL and 0?

NULL represents the complete absence of data, an unknown value, or a missing entry in a database. It does not occupy space as a value and represents "nothingness". In contrast, 0 is a concrete numerical value representing the integer zero. For example, a bank balance of 0 means there is no money in the account, whereas a balance of NULL means the account balance is unknown or unrecorded.

2. What is a default constraint?

A DEFAULT constraint is used to set a default value for a column when no explicit value is specified during an INSERT statement. This ensures data consistency and prevents unintended NULL values in columns where a standard initial value makes sense (e.g., setting a default `status = 'Active'` or `created_at = CURRENT_TIMESTAMP`).

3. How does IS NULL work?

In SQL, standard comparison operators like `=`, `!=`, or `<>` cannot be used to compare values with NULL because `NULL = NULL` evaluates to UNKNOWN (Three-Valued Logic). The `IS NULL` operator (and its counterpart `IS NOT NULL`) specifically tests whether an expression or column contains an absent/missing value.

4. How do you update multiple rows?

Multiple rows can be updated by issuing an UPDATE statement with a WHERE clause that matches more than one record. Additionally, multiple columns can be updated simultaneously by separating column assignment pairs with commas.

5. Can we insert partial values?

Yes, you can insert partial values by explicitly specifying the column names in the INSERT INTO statement. Columns omitted from the statement will automatically take their default value (if defined) or NULL (if nullable).

6. What happens if a NOT NULL field is left empty?

If you attempt to insert a record without providing a value for a column defined with a NOT NULL constraint (and no default value is defined), the database engine will reject the operation and throw an integrity constraint violation error (e.g., NOT NULL constraint failed).

7. How do you rollback a deletion?

To rollback a deletion, the DELETE statement must be executed within an explicit transaction block. By executing BEGIN TRANSACTION; before the deletion, you can issue a ROLLBACK; command if needed. Once a transaction is finalized using COMMIT;, the deletion cannot be rolled back via standard transaction controls.

8. Can we insert values into specific columns only?

Yes. By specifying the target column names in parentheses after the table name, e.g., INSERT INTO TableName (Col1, Col2) VALUES (Val1, Val2);, values are inserted exclusively into those columns. Unspecified columns will inherit default values or NULL.

9. How to insert values using SELECT?

Values can be inserted from another table using the INSERT INTO ... SELECT statement. This allows querying data from one or more source tables and directly populating a target table without manually supplying literal values.

10. What is ON DELETE CASCADE?

ON DELETE CASCADE is a foreign key constraint action. When a parent record in a primary table is deleted, all corresponding referencing records in child tables are automatically deleted by the database system, maintaining referential integrity without requiring manual cleanups.

Part 2: SQL Deliverables & Practical Execution Script

The following SQL script demonstrates Schema Creation, Data Insertion with Nulls/Defaults, Data Modification via UPDATE, Data Cleaning via DELETE, and Query Verification.

SQL IMPLEMENTATION SCRIPT (TASK_2_DATA_HANDLING.SQL)

```
-- =====
-- Elevate Labs SQL Developer Internship - Task 2
-- Student Name: G. Pavana Deepika
-- Topic: Data Insertion, Handling Nulls, Updates, and Deletions
-- Compatible with: SQLite / DB Fiddle
-- =====

-- 1. Table Creation with Constraints
DROP TABLE IF EXISTS employees;

CREATE TABLE employees (
    emp_id INTEGER PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE,
    department VARCHAR(50) DEFAULT 'Unassigned',
    salary DECIMAL(10,2),
    joining_date DATE DEFAULT CURRENT_DATE
);

-- 2. Insert Complete Rows (FULL Values)
INSERT INTO employees (emp_id, first_name, last_name, email, department, salary,
joining_date)
VALUES
(1, 'Pavana', 'Deepika', 'pavana.deepika@example.com', 'Data Analytics', 75000.00, '2026
```

```

-01-15'),
(2, 'Alex', 'Smith', 'alex.smith@example.com', 'Engineering', 68000.00, '2025-11-20');

-- 3. Insert Partial Values (Demonstrating DEFAULT and NULL behaviors)
INSERT INTO employees (emp_id, first_name, last_name, email)
VALUES
(3, 'Rahul', 'Sharma', 'rahul.sharma@example.com');
-- Result: department becomes 'Unassigned', salary becomes NULL

INSERT INTO employees (emp_id, first_name, last_name, department, salary)
VALUES
(4, 'Priya', 'Verma', 'HR', NULL);
-- Result: email becomes NULL, salary is explicitly NULL

-- 4. Insert Data using SELECT statement (Table Migration Simulation)
CREATE TABLE new_recruits (
    id INTEGER,
    fname VARCHAR(50),
    lname VARCHAR(50),
    mail VARCHAR(100),
    dept VARCHAR(50),
    pay DECIMAL(10,2)
);

INSERT INTO new_recruits VALUES
(5, 'Michael', 'Brown', 'mbrown@example.com', 'Engineering', 72000.00),
(6, 'Sara', 'Connor', 'sara.c@example.com', 'Security', 0.00);

INSERT INTO employees (emp_id, first_name, last_name, email, department, salary)
SELECT id, fname, lname, mail, dept, pay FROM new_recruits;

-- 5. Using IS NULL and IS NOT NULL to query missing data
-- Query records with missing salary
SELECT * FROM employees WHERE salary IS NULL;

-- 6. UPDATE Statement: Update single and multiple rows
-- Assign missing salary for unassigned/null profiles
UPDATE employees
SET salary = 50000.00, department = 'Operations'
WHERE salary IS NULL;

-- Give a 10% raise to Engineering department (Multiple row update)
UPDATE employees
SET salary = salary * 1.10
WHERE department = 'Engineering';

-- 7. DELETE Statement with WHERE Clause and Transaction Handling
BEGIN TRANSACTION;

-- Delete employee records with 0 or negative salary
DELETE FROM employees
WHERE salary <= 0;

-- Confirm changes
COMMIT;

-- Final State Check
SELECT * FROM employees;

```